

INTRANET 2 - Actualites et BDD

Auteur

Nom : [DEMAISON Camille]

Dans cette deuxième partie on a mis en place le système de publication sur notre page d'accueil. Nous avons fait en sorte que les actualités soient visibles sur la page d'accueil du site et soient classées par date de publication.

Pour gérer ce système d'actualités nous avons mis en place une base de données ce qui a permis de stocker les publications. Sans cela si nous avions utilisé une liste dans le code alors cela serai disparu à chaque arrêt du code et puis ça permet aussi de pouvoir gérer des quantités d'informations plus facilement.

Structure du projet :

Actuellement l'application est situé dans un dossier nommé `/opt/flask_app/` avec cette structure :

```
/opt/flask_app/
├─ app.py                # Déjà présent
├─ requirements.txt      # Déjà présent
├─ templates/
│   ├─ login.html        # Déjà présent
│   ├─ home.html         # Déjà présent
│   └─ publish.html      # Page de publication
├─ static/
│   └─ favicon.ico       # Déjà présent
└─ instance/             # Dossier créé automatiquement lors du
    premier post
    └─ news.db           # Pareil pour la BDD
```

Technologies utilisées :

Voici le tableau mis à jour avec les bibliothèques utilisées actuellement :

Technologie	Usage
Flask	Framework web principal
Flask-Login	Gestion des sessions utilisateurs
Flask-WTF	Gestion sécurisée des formulaires avec CSRF intégré
Flask-SQLAlchemy	ORM pour interagir avec la base de données
flask_ldap3_login	Authentification via LDAP/LDAPS
ldap3	Connexion et opérations LDAP avec support SSL/TLS
WTForms	Définition des champs et validation des formulaires Pythonic
Bootstrap 5	Interface utilisateur moderne et réactive
Bleach	Filtrage des balises d'un texte

Bibliothèques installées

On a actuellement besoin d'installer ces bibliothèques pour faire fonctionner cette partie du projet :

```
Flask
Flask-Login
Flask-WTF                # Présent dans flask-ldap3-login mais
utilisé autrement ici
Flask-SQLAlchemy         # Nouvelle bibliothèque
flask-ldap3-login
ldap3
wtforms                  # Nouvelle bibliothèque
bleach                   # Nouvelle bibliothèque
```

app.py — Application principale

On a du apporter des **modifications** dans `app.py` pour intégrer la base de données, le modèle des actualités et les routes associées. Pour gérer les bases de données nous avons utilisé un module de Flask [SQLAlchemy](#).

Importation des bibliothèques

Dans un premier temps on importe tout les bibliothèques supplémentaires nécessaires pour faire fonctionner cette partie du projet.

```
from flask_sqlalchemy import SQLAlchemy
from flask_wtf import FlaskForm
from wtforms import StringField, TextAreaField, SubmitField
from wtforms.validators import DataRequired
from datetime import datetime

import bleach
```

Info

`bleach` est une librairie maintenue par Mozilla, conçue pour désinfecter le HTML (et elle est très légère).

Filtrage des balises

On défini une fonction de filtrage des balises pour empêcher les attaques XSS, on échappe tout HTML dangereux.

```
def sanitize_text(text):
    return bleach.clean(text, tags=[], strip=True) # Supprime toutes les
    balises HTML
```

On crée cette fonction comme ça on pourra l'utiliser sur n'importe quelle route et l'améliorer plus facilement si besoin. Avant c'était géré par Flask directement mais là ce n'est pas le cas car un des champs des posts ne sera plus une `String` littérale (utilisation du filtre `safe` pour utiliser les retour à la ligne).

Configuration de la base de données

On ajoute cette ligne juste après `app.config['DEBUG'] = True` :

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///news.db'
db = SQLAlchemy(app)
```

- `SQLALCHEMY_DATABASE_URI = 'sqlite:///news.db'` => Indique à SQLAlchemy où se trouve la base de données

`'sqlite:///news.db'` : Cela signifie : “Utilise SQLite et stocke les données dans un fichier local appelé `news.db`.”

- `sqlite://` : signifie qu'on utilise SQLite comme moteur de base de données.
- `news.db` : C'est le nom du fichier de la base de données qui sera créé dans le répertoire courant.

Après cela on initialise le module SQLAlchemy.

Création du modèle d'actualité

On a ajouté ce bloc après la classe `User` :

```
class News(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(255), nullable=False)
    content = db.Column(db.Text, nullable=False)
    published_at = db.Column(db.DateTime, default=datetime.utcnow)
```

Ici on déclare la classe `News` :

- `News` : C'est le nom du modèle, donc aussi celui de la **table** dans la base de données (elle s'appellera par défaut `news`, en minuscule).
- `db.Model` : C'est la classe de base de SQLAlchemy dont tous les modèles doivent hériter. Elle fournit des fonctionnalités pour interagir avec la base.

On déclare ensuite la **structure de la base de données** dans la classe. On a **4 éléments** qui sont :

Colonne `id`

```
id = db.Column(db.Integer, primary_key=True)
```

- `db.Column` : C'est une colonne dans la table.
- `db.Integer` : Le type de données est un entier.
- `primary_key=True` : C'est la **clé primaire** de la table, donc chaque news aura un `id` **unique et auto-incrémenté**.

Colonne `title`

```
title = db.Column(db.String(255), nullable=False)
```

- `db.String(255)` : Une chaîne de caractères (maximum 255 caractères).
- `nullable=False` : Ce champ est **obligatoire**, donc une actualité doit avoir un titre.

Colonne `content`

```
content = db.Column(db.Text, nullable=False)
```

- `db.Text` : C'est pour du texte plus long que `String`, pratique pour des articles ou descriptions.
- `nullable=False` : Aussi **obligatoire**.

Colonne `published_at`

```
published_at = db.Column(db.DateTime, default=datetime.utcnow)
```

- `db.DateTime` : Le type est une date et heure.
- `default=datetime.utcnow` : Si on ne donne pas de date lors de la création d'un article, ça utilisera la date/heure actuelle automatiquement.

Cette fonctionnalité utilise `datetime`.

Résumé visuel de la table `news` :

id (int)	title (str)	content (text)	published_at (datetime)
1	"Titre"	"Blabla..."	2025-04-01 15:00:00

Formulaire Flask-WTF pour publier une actualité

On a ajouté cette classe juste après la classe `News` :

```
class NewsForm(FlaskForm):  
    title = StringField('Titre', validators=[DataRequired()])  
    content = TextAreaField('Contenu', validators=[DataRequired()])  
    submit = SubmitField('Publier')
```

Cette classe `NewsForm` définit un **formulaire web** avec Flask, en utilisant l'extension **Flask-WTF**, qui facilite la création et la validation de formulaires HTML.

- `NewsForm` : C'est le nom du formulaire.
- `FlaskForm` : On hérite de la classe de base fournie par **Flask-WTF**, qui gère la logique de formulaire (validation, CSRF vu précédemment, affichage des champs, etc.).

Warning

Pour que ça fonctionne, on doit avoir importé `FlaskForm`, même si `flask-ldap3-login` est importé car cette librairie l'utilise mais on va utiliser `FlaskForm` différemment.

On déclare ensuite la **structure du formulaire** dans la classe. On a **3 éléments** qui sont :

Champ title

```
title = StringField('Titre', validators=[DataRequired()])
```

- `StringField` : C'est un champ de texte classique (équivalent à `<input type="text">`).
- `'Titre'` : C'est le **label** du champ qui s'affichera à côté du champ dans le HTML.
- `validators=[DataRequired()]` : Ce champ est **obligatoire**.
=> Si l'utilisateur soumet le formulaire vide, il y aura une erreur.

Champ content

```
content = TextAreaField('Contenu', validators=[DataRequired()])
```

- `TextAreaField` : Champ pour saisir du texte long (équivalent à `<textarea>`).
- `'Contenu'` : Label affiché.
- `DataRequired()` : Aussi **obligatoire**.

Bouton submit

```
submit = SubmitField('Publier')
```

- `SubmitField` : Un bouton pour envoyer le formulaire (équivalent à `<button type="submit">`).
 - `'Publier'` : Texte affiché sur le bouton.
-

Modification de la route / pour afficher les actualités

On a remplacé la route `@app.route('/')` par :

```
@app.route('/')
@login_required
def home():
    news_list = News.query.order_by(News.published_at.desc()).all()
    return render_template('home.html', user=current_user,
                           news_list=news_list)
```

Ici on définit une **route Flask sécurisée** qui affiche la page d'accueil avec la liste des actualités. Il faut être connecté pour y accéder sinon on est redirigé vers la page de connexion.

On retrouve dans la fonction de la route deux éléments principaux qui sont :

La récupération des actualités

```
news_list = News.query.order_by(News.published_at.desc()).all()
```

- `News.query` : On interroge la table des actualités.
- `.order_by(News.published_at.desc())` : On trie par date de publication, de la **plus récente à la plus ancienne**.
- `.all()` : On récupère **toutes** les actualités (sous forme de liste Python).

On obtient une liste d'objets `News` dans la variable `news_list`, triée du plus récent au plus ancien.

Le rendu de la page HTML

```
return render_template('home.html', user=current_user,
                       news_list=news_list)
```

- `render_template('home.html')` : On affiche le fichier HTML `home.html` (dans le dossier `templates/`).
 - `user=current_user` : On passe l'utilisateur connecté à la page (grâce à Flask-Login).
 - `news_list=news_list` : On passe la liste des actualités au template pour les afficher.
-

Ajout de la route sécurisée pour publier

À la fin de `app.py`, on a ajouté :

```
@app.route('/publish', methods=['GET', 'POST'])
@login_required
def publish():
    form = NewsForm()

    if form.is_submitted():
        if form.validate():
            # Traitement securise du contenu
            clean_content = sanitize_text(form.content.data)

            news = News(title=form.title.data, content=clean_content)
            db.session.add(news)
            db.session.commit()
            flash("Actualité publiée avec succès.", "success")
            return redirect(url_for('home'))
        else:
            flash("Veuillez remplir les deux champs du formulaire.",
"error")

    return render_template('publish.html', form=form)
```

Ici on définit une route Flask qui permet aux utilisateurs authentifiés de publier une actualité via un formulaire.

Comme pour la route login on accepte deux méthodes, une pour afficher le formulaire avec 'GET' et une autre pour l'envoi avec 'POST'.

On retrouve dans la fonction de la route le même fonctionnement que pour la route de login :

- Initialisation du formulaire avec : **form = NewsForm()**
- Vérification d'un envoi d'un formulaire, si oui alors on vérifie s'il est conforme. Si ce n'est pas le cas on envoie un message d'erreur pour prévenir l'utilisateur.
- Si tout est bon alors on traite le formulaire :
 - On **nettoie le texte** pour éviter certains types d'injection
 - On crée une **nouvelle instance** de `News` (le modèle qui représente une actualité).
 - On remplit les champs `title` et `content` avec les données du formulaire.
 - On ajoute cette actualité à la base de données avec SQLAlchemy.
 - `commit()` enregistre les changements dans le fichier `news.db`.
 - Ensuite on prévient l'utilisateur que tout s'est bien déroulé puis l'utilisateur est redirigé vers la page d'accueil.

- Si l'utilisateur n'a pas envoyé de formulaire alors on lui envoie la page avec le formulaire vide

Initialisation de la base de données

On ajoute finalement ceci à la fin de notre fichier, juste avant `app.run(...)` :

```
with app.app_context():
    db.create_all()
```

Ceci permet de lier la base de données à l'application et de la créer si elle n'existe pas. (c'est une méthode safe pour gérer la BDD)

home.html (mis à jour pour afficher les actualités)

Voici les deux fichiers HTML que nous avons intégrés dans notre dossier `templates/` :

- Un template `home.html` mis à jour pour afficher les actualités.
- Un nouveau template `publish.html` pour le formulaire de publication.

On remplace donc le fichier actuel par celui-ci :

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <title>Accueil</title>
    <link rel="icon" href="{{ url_for('static', filename='favicon.ico')
  }}" type="image/x-icon">
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.
css" rel="stylesheet">
  </head>
```


Plein de nouvelles fonctionnalités ont été ajoutés.

Gestion des messages flash (alertes)

```
{% with messages = get_flashed_messages(with_categories=true) %}
  {% if messages %}
    {% for category, message in messages %}
      <div class="alert alert-{{ 'success' if category == 'success' else
'danger' }} alert-dismissible fade show" role="alert">
        {{ message }}
        <button type="button" class="btn-close" data-bs-dismiss="alert"
aria-label="Close"></button>
      </div>
    {% endfor %}
  {% endif %}
{% endwith %}
```

Ici nous allons afficher les messages "flash" qui ont été stockés. On liste tout les messages et on les affiche avec une boucle `for`.

- `get_flashed_messages()` : récupère les messages stockés temporairement, par exemple : "Post réussi".
On récupère ceux qui ont une catégorie.
- `category` permet d'afficher des messages différents : vert pour *success*, rouge sinon.
- `btn-close` : bouton pour fermer l'alerte.

Tip

Au début on avait rencontré un problème, si un message est envoyé dans la route avec la méthode `flash` alors tant qu'il n'était pas utilisé il est conservé. Il arrivait que s'il n'était pas affiché sur le coup alors il prenait la place du suivant.

Typiquement si un post de formulaire était réussi mais que l'on n'affiche pas le message puis qu'on se déconnecte alors sur la page de connexion apparaissait :
Actualité publiée avec succès.

Bloc de bienvenue

On a aussi modifié le bloc de bienvenue pour être plus agréable.

```
<div class="card shadow-sm mb-4">
  <div class="card-body">
    <h2 class="card-title">Bienvenue {{ user.data.cn }}</h2>
    <p class="card-text">Ce site est un forum de publication d'actualité.
    Voici les posts des salariés d'ITWay.</p>
    <a href="{{ url_for('logout') }}" class="btn btn-danger">Se
    déconnecter</a>
    <a href="{{ url_for('publish') }}" class="btn btn-success ms-
    2">Publier une actualité</a>
  </div>
</div>
```

Pour cela nous avons utilisé divers éléments tels que :

- L'affichage d'un message personnalisé avec le nom de l'utilisateur connecté ({{ user.data.cn }}) et un paragraphe de bienvenue.
- Deux boutons : un pour se déconnecter, l'autre pour publier une actu. `url_for('logout')` et `url_for('publish')` génèrent les bonnes URL Flask pour ces routes.

Affichage des actualités

```
{% for news in news_list %}
  <div class="card shadow-sm mb-3">
    <div class="card-body">
      <h4 class="card-title">{{ news.title }}</h4>
      <p class="card-text">{{ news.content | replace('\n', '<br>') | safe
      }}</p>
      <small class="text-muted">Publié le {{
      news.published_at.strftime('%d/%m/%Y à %H:%M') }}</small>
    </div>
  </div>
{% else %}
  <p>Aucune actualité pour le moment.</p>
{% endfor %}
```

Ici on utilise une boucle for pour afficher tout les actualités présentes dans la BDD.

- Pour chaque actu dans `news_list`, on affiche :
 - Le **titre**
 - Le **contenu**
 - | `replace('\n', '<br>')` => Remplace les retour à la ligne par des `<br>` car sinon ils ne seront pas pris en compte.
 - | `safe` => Autorise le HTML dans le texte puisqu'il a été filtré au préalable (permet d'afficher les `<br>`)
 - La **date de publication** formatée sous le modèle français (JJ/MM/YYYY à HH:MM)
 - Si aucune actu => Affiche "Aucune actualité pour le moment"
-

Scripts JS de Bootstrap

```
<script  
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
```

Nécessaire pour faire fonctionner les composants interactifs de Bootstrap (comme ici les alertes fermables, le menu burger, etc.).

Info

Evolution possible : Supprimer un post

On pourrait ajouter la fonctionnalité de suppression de post en ajoutant une croix sur les post seulement visible par un utilisateur de type `admin`.

Pour cela il faudrait ajouter une route `/delete` qui serait autorisé seulement aux utilisateurs authentifiés de type `admin` et qui supprimerait l'entrée dans la BDD. Elle serait utilisé lors de l'appui sur la croix puis redirigerait vers `home`.

publish.html

On a ajouté ce nouveau fichier dans templates/ :

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <title>Publier une actualité</title>
    <link rel="icon" href="{{ url_for('static', filename='favicon.ico')
    }}" type="image/x-icon">

    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.
css" rel="stylesheet">

  </head>
  <body class="bg-light">
    <div class="container mt-5">

      {% with messages = get_flashed_messages(with_categories=true) %}
      {% if messages %}
        {% for category, message in messages %}
          <div class="alert alert-{{ 'success' if category == 'success'
else 'danger' }} alert-dismissible fade show" role="alert">
            {{ message }}
            <button type="button" class="btn-close" data-bs-
dismiss="alert" aria-label="Close"></button>
          </div>
        {% endfor %}
      {% endif %}
      {% endwith %}

      <div class="card shadow-sm">
        <div class="card-body">
          <h3 class="card-title">Nouvelle actualite</h3>
          <form method="POST">
            {{ form.hidden_tag() }}
            <div class="mb-3">
              {{ form.title.label(class="form-label") }}
              {{ form.title(class="form-control") }}
            </div>
            <div class="mb-3">
              {{ form.content.label(class="form-label") }}
              {{ form.content(class="form-control", rows=5) }}
            </div>
            <div class="d-grid">
              {{ form.submit(class="btn btn-primary") }}
```

```

        </div>
    </form>
    <a href="{{ url_for('home') }}" class="btn btn-link mt-3">Retour
a l'accueil</a>

    </div>
</div>
</div>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle
.min.js"></script>

</body>
</html>

```

Cette page utilise le format de **WTForms** pour gérer le formulaire côté Flask.

Gestion des messages flash (alertes)

Cette partie a le même fonctionnement que dans la page home.html.

```

{% with messages = get_flashed_messages(with_categories=true) %}
    {% if messages %}
        {% for category, message in messages %}
            <div class="alert alert-{{ 'success' if category == 'success' else
'danger' }}" alert-dismissible fade show" role="alert">
                {{ message }}
                <button type="button" class="btn-close" data-bs-dismiss="alert"
aria-label="Close"></button>
            </div>
        {% endfor %}
    {% endif %}
{% endwith %}

```

Formulaire de publication

```

<div class="card shadow-sm">
    <div class="card-body">
        <h3 class="card-title">Nouvelle actualite</h3>
        <form method="POST">

```

- Utilise un **card Bootstrap** pour styliser le formulaire.
 - `method="POST"` : les données seront envoyées au serveur via la méthode POST.
-



Protection CSRF (Cross-Site Request Forgery)

```
{{ form.hidden_tag() }}
```

On utilise un token CSRF comme lors de la connexion pour sécuriser le formulaire.

Zone d'entrée par l'utilisateur

On retrouve plusieurs parties qui permettent de remplir le formulaire.

Champ "Titre"

```
<div class="mb-3">
  {{ form.title.label(class="form-label") }}
  {{ form.title(class="form-control") }}
</div>
```

On crée un champ pour le titre avec ces éléments :

- `form.title.label` => Le label du champ (généré automatiquement)
- `form.title` => Le champ de saisie
- `form-control` => Classe Bootstrap pour styliser les inputs

Champ "Contenu"

```
<div class="mb-3">
  {{ form.content.label(class="form-label") }}
  {{ form.content(class="form-control", rows=5) }}
</div>
```

Même logique, sauf que `form.content` est un `TextAreaField`, donc avec plusieurs lignes (`rows=5`).

Bouton "Publier"

```
<div class="d-grid">
  {{ form.submit(class="btn btn-primary") }}
</div>
```

- `d-grid` permet d'étirer le bouton sur toute la largeur.
 - `btn btn-primary` = bouton bleu Bootstrap.
 - `form.submit` est le bouton généré automatiquement par WTForms.
-

Lien pour retourner à l'accueil

```
<a href="{{ url_for('home') }}" class="btn btn-link mt-3">Retour a  
l'accueil</a>
```

- Lien stylisé comme un bouton pour revenir à la page d'accueil.
 - `url_for('home')` appelle la route Flask nommée `'home'`.
-

Scripts Bootstrap JS

```
<script  
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle  
.min.js"></script>
```

Comme pour la page `home.html`, permet de faire fonctionner les éléments interactif de Bootstrap.

Tester l'application Flask

Pour tester l'application il ne reste plus qu'à faire comme précédemment :

```
apt update && apt install python3-pip -y

pip install --ignore-installed --no-cache-dir -r
/opt/flask_app/requirements.txt --break-system-packages
```

Puis pour lancer l'application :

```
python3 /opt/flask_app/app.py
```

Il ne reste plus qu'à tester sur un navigateur web d'une autre machine du réseau via :
`http://srv-intranet:5000`.

Tests et vérifications

Comme l'utilisateur peut entrer des valeurs dans le formulaire il va falloir vérifier qu'il ne puisse pas extraire des informations ou casser le site. En effet si une personne malveillante arrive à contourner l'identification il pourrait faire en sorte de bloquer l'utilisation des autres utilisateurs.

Voici un exemple des différents tests effectués :

Bienvenue Camille Demaison

Ce site est un forum de publication d'actualité. Voici les posts des salariés d'ITWay.

[Se déconnecter](#) [Publier une actualité](#)

```
<script>alert("XSS");</script>
```

alert("XSS");

Publié le 05/04/2025 à 15:57

```
{{7*7}}
```

{{7*7}}

Publié le 05/04/2025 à 15:56

```
'; DROP TABLE News; --
```

'; DROP TABLE News; --

Publié le 05/04/2025 à 15:55

Test

Paragraphe
avec
des
lignes.

Publié le 05/04/2025 à 15:55

Différents tests ont été procédé :

- Vérification des BDD :

Pour vérifier l'état de la BDD créé après plusieurs posts on va utiliser un outils appelé `sqlite3`.

On l'installe avec :

```
apt install sqlite3
```

Il suffit ensuite de se placer dans le dossier contenant la BDD puis de lancer l'outils sur notre fichier :

```
cd /opt/flask_app/instance
```

```
sqlite3 news.db
```

```
sqlite> .headers on
```

```
sqlite> .mode column
```

```
sqlite> SELECT * FROM news;
```

id	title	content	published_at
1	Test	Paragraphe	2025-04-05
15:55:30.464240		avec des lignes.	
2	' ; DROP TABLE News; --	' ; DROP TABLE News; --	2025-04-05
15:55:51.419976			
3	{{7*7}}	{{7*7}}	2025-04-05
15:56:54.086929			
4	<script>alert("XSS");</script>	alert("XSS");	2025-04-05
15:57:11.409068			

Cela affiche la table avec son contenu. (utilisation de filtres pour rendre l'affichage moins brut)

On constate que les données entrées sont bien enregistré dans la BDD et que l'intégrité des données est bien conservée.

- Protection contre les injections SQL :

Pour tester cela on a du publier un post contenant une injection du style : (on place cela dans le titre et le contenu)

```
' ; DROP TABLE News ; --
```

On constate que l'affichage de la publication est normal et n'affiche pas la table complète dans le post.

Cela est possible car on utilise un module de `Flask-SQLAlchemy` qui protège lui même contre ce type d'injection.

- Protection contre SSTI :

Pour tester cela on a du publier un post contenant une injection du style : (on place cela dans le titre et le contenu)

```
{{7*7}}
```

On constate que l'affichage de la publication est normal et pas 49 malgré l'utilisation du filtre `safe` pour le contenu du post.

Cela est possible car on utilise un module de `Flask-WTF` et `WTForms` qui protègent eux-même contre ce type d'injection.

- Protection contre XSS :

Pour tester cela on a du publier un post contenant une injection du style : (on place cela dans le titre et le contenu)

```
<script>alert("XSS");</script>
```

On constate que l'affichage de la publication est normal.

Cela est possible car on a créé une fonction `sanitize_text(text)` qui protège contre ce type d'injection. En effet elle supprime toutes les balises (HTML, JS, etc.) qui seraient présente dans le texte.